UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Construcción de Compiladores

Ing. Pablo Koch



Proyecto - Compiscript

Compilador de Compiscript a MIPS en Rust: de la gramática al ensamblador

Ricardo Godínez - 23247

Vianka Castro - 23201

Javier André Benitez García - 23405

GUATEMALA, 7 de noviembre de 2026

Índice

1. Introducción 3

2. Objetivos 3

3. Arquitectura general 4

4. Análisis léxico 5

5. Análisis sintáctico 7

6. Análisis semántico 7

7. Código intermedio 9

8. Generación de código MIPS 10

9. Pruebas 15

10. Discusión 16

11. Conclusiones 16

12. Referencias 17

1. Introducción

Este informe resume el compilador que construimos durante el semestre para **Compiscript**, un lenguaje pequeño con tipos, funciones anidadas, clases con herencia, listas y manejo de excepciones con `try`/`catch`. El compilador recibe un archivo `.cps` y lo lleva por todas las fases clásicas del libro del dragón: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio y, al final, generación de código ensamblador MIPS que se ejecuta en el simulador MARS.

Desde el inicio tomamos una decisión que marcó todo el proyecto: en lugar de usar ANTLR, construimos nuestro propio generador de analizadores léxicos (a partir de un archivo `.yal`, al estilo de YALex) y nuestro propio generador de analizadores sintácticos LALR(1) (a partir de un archivo `.yalp`, al estilo de YAPar). Todo está escrito en Rust, y encima pusimos una API HTTP y un IDE web donde se escribe un programa, se compila y se ve cada fase por separado.

Hacemos un recorrido corto por las primeras cuatro fases y nos detenemos en la última, la traducción a MIPS, que fue la entrega más reciente y la que más decisiones de diseño nos pidió: el registro de activación, la secuencia de llamada, la asignación de registros con `getReg()`, las rutinas de apoyo y la ejecución en MARS desde las pruebas, la consola y el IDE.

El código completo está en el repositorio del equipo: **https://github.com/Vann06/Proyecto_Lexer_Gen**. Cada integrante trabajó en su propia rama y con sus propios commits.

2. Objetivos

Objetivo general

Construir un compilador completo para Compiscript que traduzca un programa fuente a ensamblador MIPS ejecutable, reportando con línea y columna los errores de cada fase.

Objetivos específicos

- Generar el analizador léxico de Compiscript a partir de expresiones regulares, con los algoritmos de Thompson, construcción de subconjuntos y minimización.
- Generar un analizador sintáctico LALR(1) sin conflictos a partir de la gramática del lenguaje y construir el árbol sintáctico.

- Implementar el análisis semántico con tabla de símbolos por ámbitos, sistema de tipos, clases, funciones y control de flujo.
- Traducir el árbol a código de tres direcciones (TAC) con registros de activación y un intérprete que lo ejecute.
- Traducir el TAC a MIPS respetando el registro de activación, con secuencias de llamada y retorno y asignación de registros mediante getReg().
- Ejecutar el ensamblador generado en MARS desde las pruebas automáticas, la consola y el IDE, y detectar ciclos infinitos y desbordes de memoria.

3. Arquitectura general

El compilador es una cadena de fases donde cada una recibe la salida de la anterior. La Tabla 1 resume qué entra y qué sale de cada fase y en qué módulo de Rust vive. Todas las fases se ejecutan desde una sola función del backend (`api::pipeline`), que comparten la API, la consola (`cpsc`) y las pruebas, así que lo que vemos en el IDE es exactamente lo que prueban los tests.

Tabla 1. *Fases del compilador, su entrada, su salida y el módulo donde están.*

Fase	Recibe	Entrega	Módulo
Léxica	compiscript.yal y el fuente .cps	Lista de tokens con línea y columna	src/lexico
Sintáctica	compiscript.yalp y los tokens	Árbol sintáctico (o errores con lo esperado)	src/sintactico
Semántica	Árbol sintáctico y directivas de la gramática	Tabla de símbolos, tipos y diagnósticos	src/semantico
Intermedia	Árbol anotado y tabla de símbolos	Programa TAC con marcos y vtables	src/intermedio
Código objetivo	Programa TAC	Ensamblador MIPS (.asm) y estadísticas de registros	src/codigo_objetivo
Ejecución	El .asm	Salida de MARS y código de salida	codigo_objetivo/sim.rs

Alrededor del compilador hay tres formas de usarlo: la consola `cpsc` (que con la opción de simulación compila un `.cps`, escribe el `.asm` y lo ejecuta en MARS), el servidor HTTP hecho con Axum y el IDE-lite en React. Los dos últimos se levantan con Docker Compose; el contenedor de la API incluye Java y MARS, así que el botón de ejecutar del IDE corre el

programa de verdad en el simulador.

La Figura 1 muestra el IDE con `rubrica.cps` cargado, el programa de prueba que cubre los puntos de la rúbrica. A la izquierda están los archivos del workspace, al centro el editor y a la derecha el panel de resultados con una pestaña por fase.



Figura 1. *IDE-lite con rubrica.cps analizado en modo LALR(1). El panel derecho muestra el árbol sintáctico.*

4. Análisis léxico

La primera fase fue nuestro generador de analizadores léxicos. El archivo `compiscript.yal` define macros con `let` (dígitos, letras, identificadores, literales, comentarios) y una lista de 59 reglas, cada una con la expresión regular de un token y el nombre que devuelve. A partir de eso el generador sigue los pasos del libro: expande las macros, convierte cada expresión regular en un árbol, arma un AFN con el algoritmo de Thompson (que construye el autómata pegando piezas pequeñas, una por operador), lo convierte en AFD con la construcción de subconjuntos y lo minimiza. Para Compiscript el AFD mínimo quedó con 153 estados.

El AFD se guarda como una tabla de transiciones `delta[estado][carácter]`, y el simulador la recorre con la regla del token más largo (maximal munch): sigue avanzando mientras haya transición y, al trabarse, regresa al último estado de aceptación que vio. Si dos reglas aceptan el mismo lexema gana la que aparece primero, por eso las palabras reservadas van antes que los

identificadores. El generador también puede emitir el lexer como un archivo `.rs` independiente.

En esta última etapa nos topamos con un detalle de esta fase: los comentarios se cortaban en el primer acento o símbolo, porque la regla solo aceptaba letras y unos pocos signos, y la tabla solo tenía columnas para los 128 caracteres ASCII. Cambiamos la regla para que un comentario llegue hasta el fin de línea, como en `Compiscript.g4`, y agregamos a la tabla una columna 129 que representa a todos los caracteres no ASCII. Solo la usan las clases negadas (`[^...]`), así que un `ñ` dentro de un comentario es válido, pero fuera de él sigue siendo un error léxico.

La Figura 2 muestra la pestaña TOKENS: los 534 tokens de `rubrica.cps`, cada uno con su tipo, su lexema, su línea y su columna.

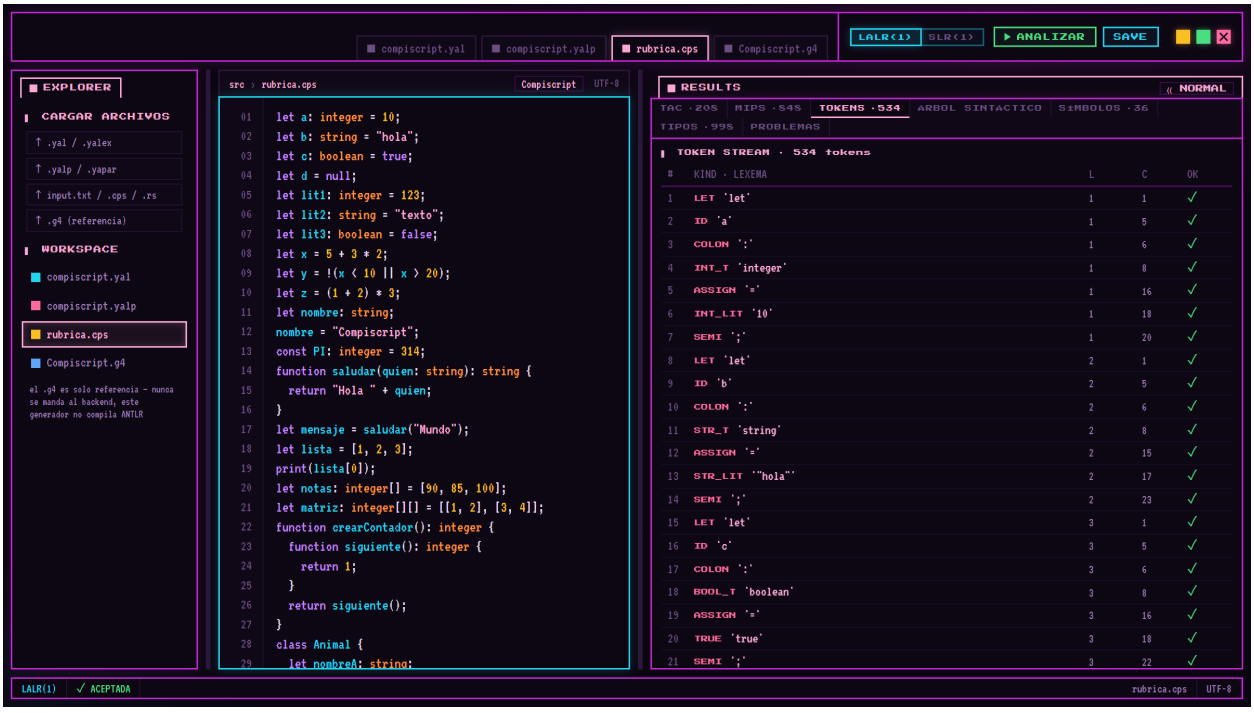


Figura 2. *Flujo de tokens que produce el analizador léxico para rubrica.cps.*

5. Análisis sintáctico

Con los tokens listos pasamos al generador de analizadores sintácticos. La gramática de Compiscript vive en `compiscript.yalp`: 60 terminales, 54 no terminales y 134 producciones, con la precedencia de operadores escrita como niveles de la gramática (`or_expr`, `and_expr`, `equality_expr`, hasta `unary` y `primary`). El generador calcula FIRST y FOLLOW, construye la colección de ítems LR(1) y fusiona los estados con el mismo núcleo para obtener LALR(1). La gramática quedó con **264 estados y ningún conflicto**. El generador también soporta SLR(1) y LL(1), que usamos en entregas anteriores.

El parser es un autómata de pila dirigido por la tabla ACTION/GOTO. Para `rubrica.cps` hace 1792 pasos entre desplazamientos y reducciones, y cada reducción arma un nodo del árbol sintáctico, que el IDE dibuja con Graphviz (Figura 1).

Donde más trabajamos al final fue en los mensajes de error. Al fusionar estados, LALR anuncia como esperados tokens que en realidad no caben ahí, y los mensajes salían con listas enormes. Lo resolvimos simulando las reducciones pendientes sobre la pila real para quedarnos solo con los tokens viables, y si uno de ellos es un cierre (`;`, `)`, `]`, `}`) mostramos solo ese. Así un error se lee como "no se esperaba 'X'. Esperado: ';'". Además el parser se recupera en modo pánico, de forma que un archivo con varios errores de sintaxis los reporta todos, cada uno con su línea.

6. Análisis semántico

La fase semántica recorre el árbol y valida todo lo que la gramática no puede expresar. La pensamos para que no dependa de los nombres de la gramática: el `.yalp` marca con directivas (`%scope`, `%decl`, `%flow`, `%return`, etc.) qué producción abre un ámbito, cuál declara una variable o cuál es un ciclo, y el analizador trabaja solo con esas marcas. Así la misma implementación sirve para otras gramáticas, y lo comprobamos con una gramática de prueba distinta (Pascalito).

La tabla de símbolos guarda los entornos anidados (global, función, clase y bloque) y, para cada símbolo, su tipo, su clase y, ya pensando en las fases siguientes, su dirección: el área (`G` para globales, `fp` para parámetros y locales, `obj` para campos), el offset y el tamaño. La Figura 3 muestra esa tabla extendida en el IDE.

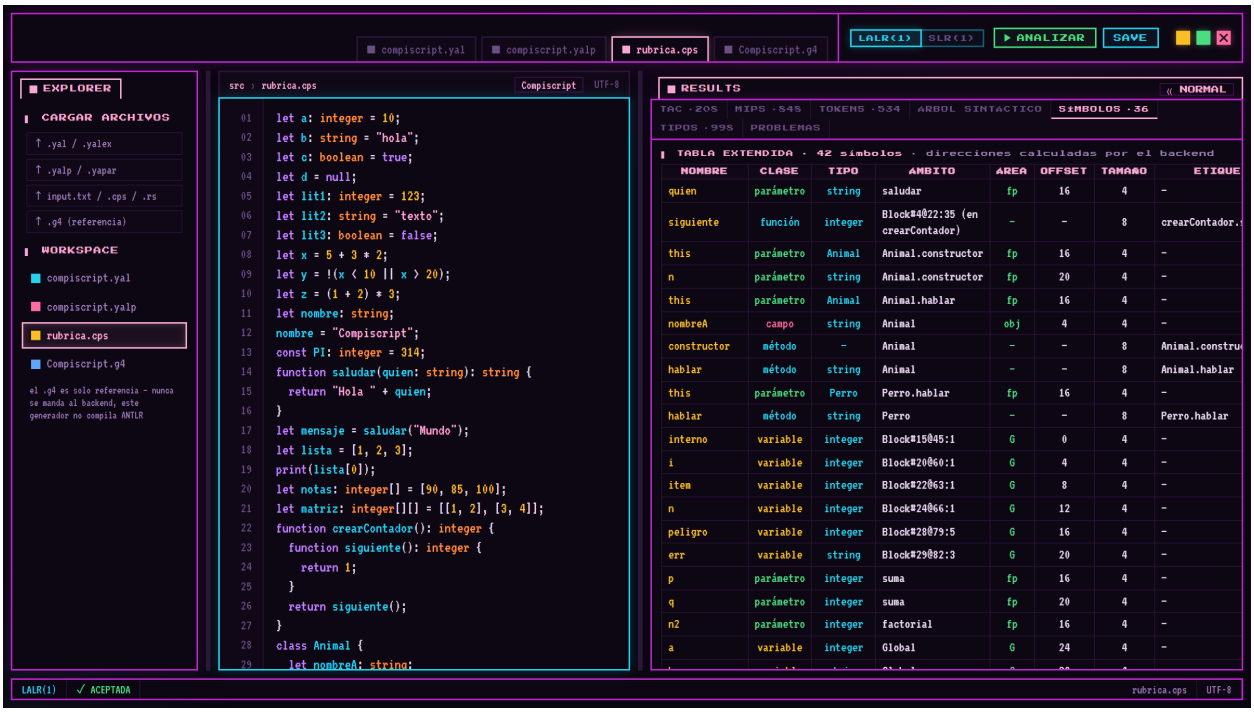


Figura 3. *Tabla de símbolos extendida de rubrica.cps, con ámbito, área, offset y tamaño de cada símbolo.*

En total el analizador produce 41 diagnósticos distintos, cada uno con código, línea y columna: tipos incompatibles, variables no declaradas, asignación a constantes, llamadas con argumentos que no coinciden, `break` fuera de un ciclo, miembros inexistentes, duplicados y código muerto, entre otros. En esta última etapa agregamos dos que encontramos al pasarle programas reales: una función con tipo de retorno en la que algún camino no tiene `return` (S017, que revisa las dos ramas de un `if`, los `switch` con `default` y los `try`), y un aviso cuando se lee una variable antes de darle un valor (W003). La Figura 4 muestra un programa con varios de estos errores a la vez.

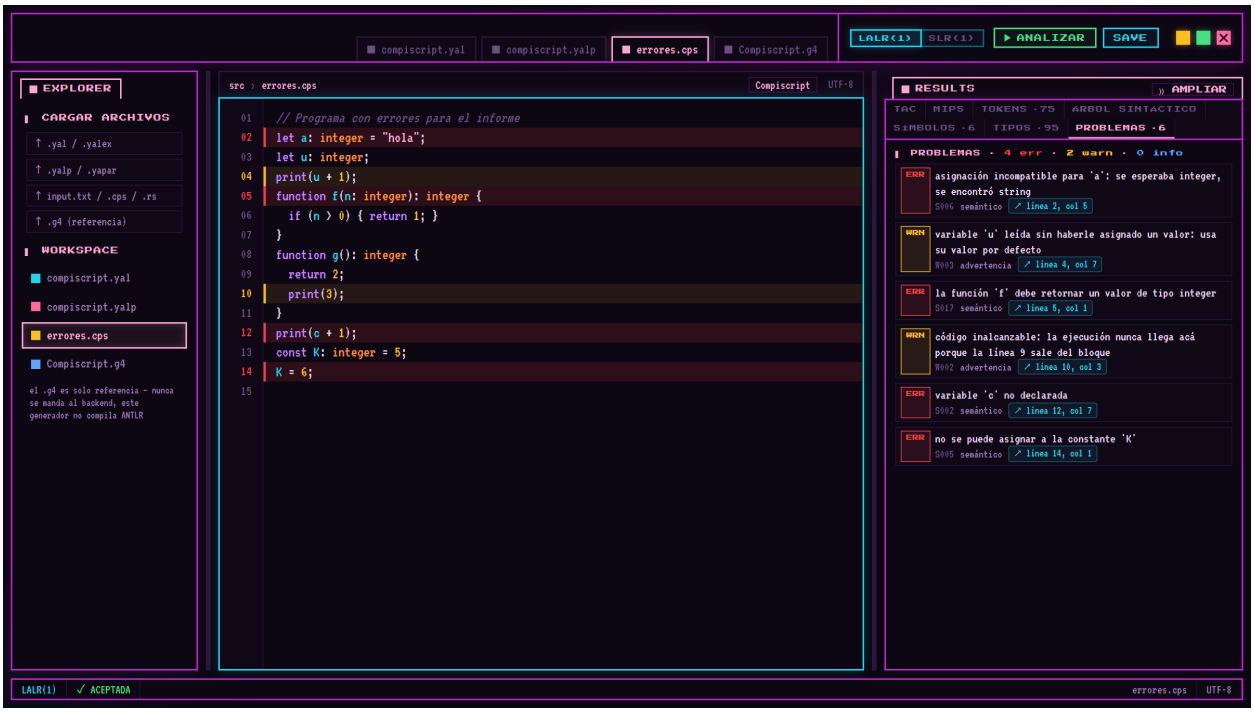


Figura 4. *Pestaña PROBLEMAS con errores semánticos y advertencias, cada uno enlazado a su línea en el editor.*

7. Código intermedio

Con el árbol validado pasamos al código de tres direcciones (TAC): instrucciones con a lo sumo un operador, del estilo `t0 = 3 * 2` o `if x < 10 goto L2`. El generador produce 23 tipos de instrucción y deja resuelto todo lo que necesita la fase siguiente. Cada variable ya trae su dirección (`G+24`, `16($fp)`), cada función su marco, y cada clase su vtable (una tabla con las direcciones de sus métodos). Los temporales se reciclan: en `main` de la rúbrica se pidieron 31 temporales, pero nunca hubo más de 4 vivos a la vez, así que el marco solo reserva 4 ranuras.

El control de flujo se traduce con etiquetas `L0`, `L1`, ... y saltos: los `if`, ciclos, `switch`, el corto circuito de `&&` y `||`, y los `break` y `continue`. Las funciones anidadas reciben un enlace de acceso (un puntero al marco de la función que las contiene) para llegar a sus variables.

Junto al generador hicimos un intérprete de TAC. Al principio era solo para probar, pero terminó siendo pieza clave de la fase de MIPS: lo ajustamos para que se comportara igual que la máquina real (enteros de 32 bits que dan la vuelta al desbordarse, mensajes de error fijos) y lo usamos como **oráculo**: la salida de MARS para un programa tiene que ser idéntica a la del intérprete. La Figura 5 muestra el TAC de la rúbrica en el IDE.

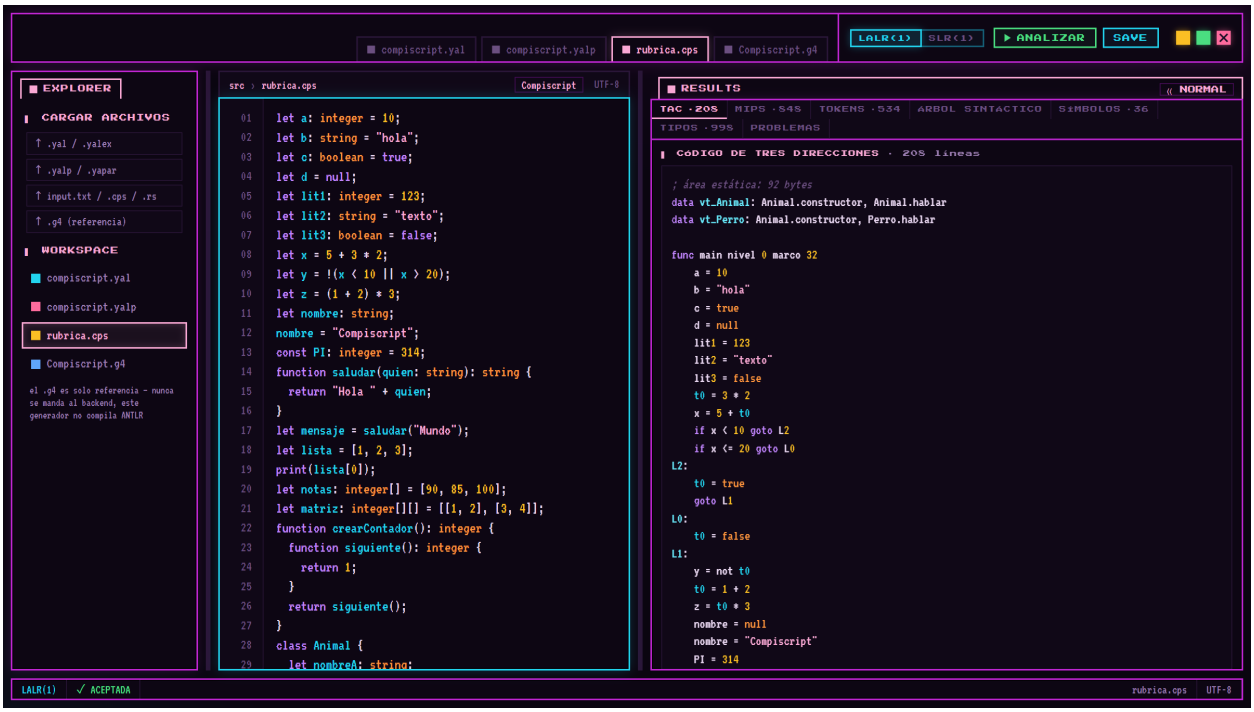


Figura 5. *Código de tres direcciones de rubrica.cps: vtables, marco de main y el corto circuito de una condición.*

8. Generación de código MIPS

La última fase traduce el TAC a ensamblador MIPS. Antes de escribir código armamos un plan entre los tres (`PLAN_MIPS.md`) con las decisiones fijas: el simulador, el uso de cada registro, el registro de activación y la secuencia de llamada exacta. Nos repartimos el trabajo por fases: Javier hizo los cimientos y la traducción de instrucciones, Vianka las secuencias de llamada, las rutinas de apoyo, el `try` y `getReg()`, y Ricardo el simulador, la consola, la API, el IDE, la batería de pruebas y el CI. Cada fase tenía definido qué recibía y qué entregaba, y eso nos dejó trabajar en paralelo sin pisarnos.

Estructura del archivo .asm

El `.asm` tiene dos secciones. En `.data` va la memoria inicial: el área de globales `G` (`.space 92` en la rúbrica, 23 variables de 4 bytes), las vtables como listas de direcciones (`.word`), los textos literales como `.asciiz` (terminados en un byte cero) y los datos del runtime. Antes de cada grupo de palabras va `.align 2`, que alinea a $2^2 = 4$ bytes, porque `lw` y `sw` fallan en una dirección que no es múltiplo de 4. En `.text` va el código: `main` primero, después cada función con el prefijo `_` (para que un nombre del usuario no choque con una instrucción de MIPS) y al final las rutinas del runtime.

Registro de activación y secuencia de llamada

El enunciado pedía manejar los parámetros, la dirección de retorno y el valor de retorno conforme al registro de activación, así que no usamos `$v0` para devolver valores: todo pasa por la pila. La Tabla 2 muestra el marco que usa cada función, con `$fp` (el frame pointer, que apunta al marco de la función que se está ejecutando) como referencia.

Tabla 2. *Registro de activación de una función, relativo a $fp.*

Offset	Contenido	Lo escribe
$fp+16 en adelante	Parámetros en orden (this primero en un método)	Quien llama
$fp+12	Enlace de acceso (funciones anidadas)	Quien llama
$fp+4	Valor de retorno	La función, en return
$fp+0	Enlace de control (el $fp de quien llama)	Quien llama
$fp-4	Dirección de retorno ($ra)	Prólogo
$fp-12 hacia abajo	Locales, ranuras de temporales y $s guardados	La función

Con ese marco, una llamada como `mensaje = saludar("Mundo")` se traduce así: quien llama reserva el registro de activación (16 bytes fijos más 4 por parámetro), escribe el parámetro y el enlace de control, salta con `jal` y, al volver, lee el valor de retorno de `4($sp)`. La función, en su prólogo, fija su `$fp` y guarda `$ra`; en el epílogo deshace todo en orden inverso.

```
# quien llama
    addiu $sp, $sp, -20     # 16 fijos + 4 por parametro
    la    $t4, S3
    sw    $t4, 16($sp)      # parametro "Mundo"
    sw    $fp, 0($sp)       # enlace de control
    jal   _saludar
    lw    $t0, 4($sp)       # valor de retorno
    addiu $sp, $sp, 20

# la funcion
_saludar:
    move  $fp, $sp          # prologo
    sw    $ra, -4($fp)
    addiu $sp, $sp, -16
    ...
    sw    $t2, 4($fp)       # return
_saludar.0ret:              # epilogo
    lw    $ra, -4($fp)
    move  $sp, $fp
    lw    $fp, 0($fp)
    jr    $ra
```

Las llamadas a métodos son virtuales: el objeto guarda en su primera palabra la dirección de la vtable de su clase, y la llamada lee la entrada del método y salta con `jalr`. Por eso `perro.hablar()` ejecuta `_Perro.hablar` aunque `Perro` herede el constructor de `Animal`.

Asignación de registros con getReg()

Primero construimos una asignación por pila, donde cada temporal vive en su ranura del marco y se carga y guarda en cada uso. Era lenta, pero correcta, y le permitió al resto del equipo avanzar sin esperar a `getReg()`. Después la reemplazamos por `getReg()` siguiendo la sección 8.6 del libro del dragón: se calcula la vida de los temporales por bloques básicos, y con un descriptor de registros (qué valor tiene cada registro) y un descriptor de direcciones (dónde está cada valor) se elige un registro para cada temporal. Un temporal que sigue vivo después de una llamada va a un registro `$s`, que se conserva entre llamadas; los demás van a `$t`. Si no queda ninguno libre, se derrama a la pila el temporal cuyo próximo uso está más lejos. Al final de cada bloque básico los valores vivos se guardan en memoria, porque el bloque siguiente no sabe en qué registro quedaron.

El `.asm` deja comentada cada decisión (`# t0 -> $s0`) y la API devuelve una tabla por función con los registros usados, los derrames y los `$s` guardados. La Tabla 3 compara las dos asignaciones sobre `rubrica.cps`: con `getReg()` el programa hace 65 lecturas y 29 escrituras de memoria menos, y la salida en MARS es la misma.

Tabla 3. *Accesos a memoria en el .asm de rubrica.cps según la asignación de registros.*

Asignación	Instrucciones lw	Instrucciones sw
Por pila (opción de cpsc)	158	137
getReg() (por defecto)	93	108

La Figura 6 muestra la pestaña MIPS después de ejecutar la rúbrica en MARS: arriba la salida del programa con código de salida 0 y abajo el ensamblador con resaltado. La Figura 7 muestra la tabla de registros por función, donde ninguna función necesitó derramar.

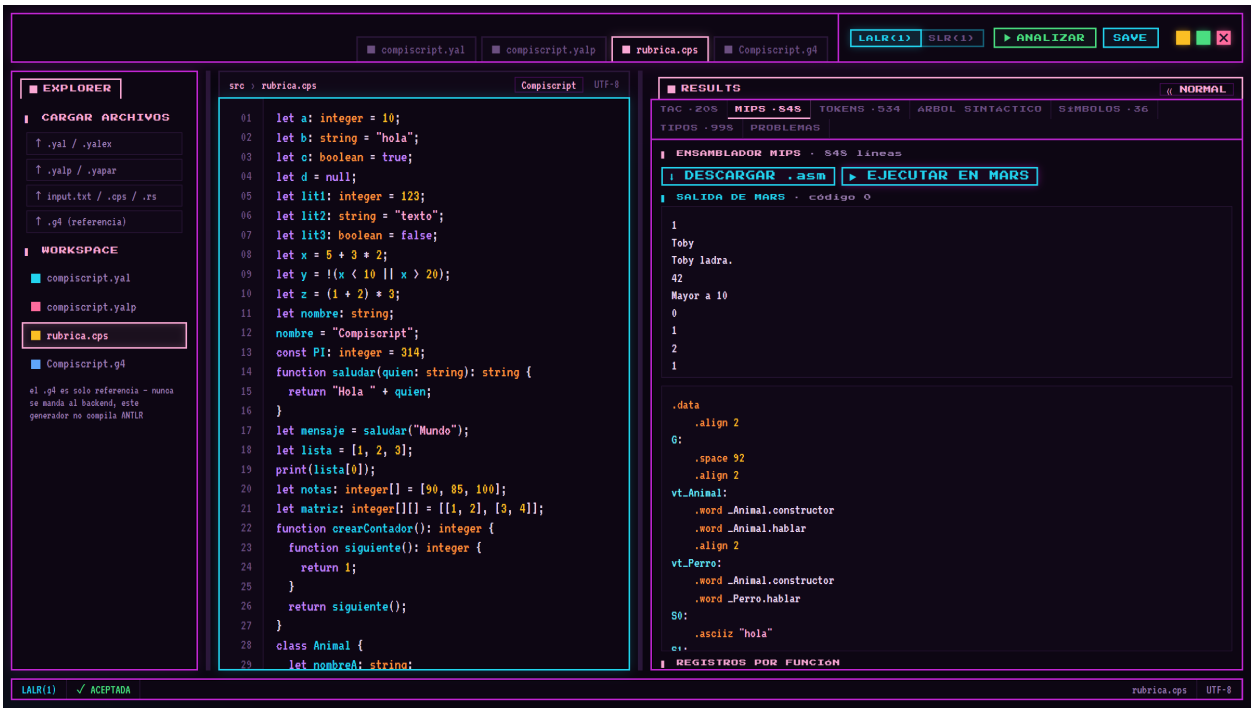


Figura 6. *Pestaña MIPS: salida de MARS para rubrica.cps (código 0) y el inicio de la sección .data.*

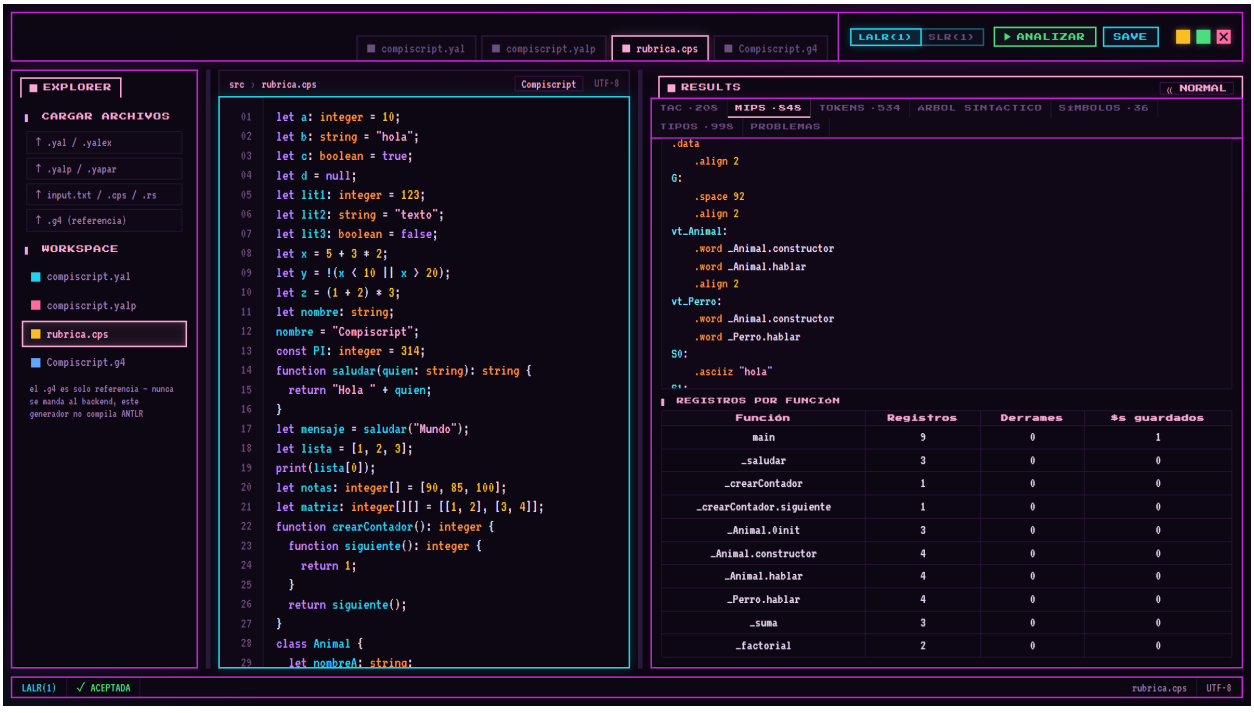


Figura 7. *Tabla de registros por función que calcula getReg() para rubrica.cps.*

Rutinas de apoyo y excepciones

MIPS no trae cosas que el TAC da por hechas, así que escribimos un runtime en ensamblador: imprimir enteros y textos (`syscall` 1 y 4), pedir memoria al heap (`syscall` 9), concatenar textos, revisar los límites de una lista y la división por cero. Estas rutinas solo tocan `$a0` a `$a3`, `$v0`, `$v1`, `$t8` y `$t9`, que `getReg()` nunca asigna, así que quien las llama no tiene que guardar nada.

Para `try`/`catch`, `try_begin` apila un registro de 48 bytes con el `try` anterior, la dirección del `catch`, `$fp` y los registros `$s0` a `$s7`. Cuando una rutina detecta un error, revisa si hay un `try` activo: si lo hay, restaura el estado guardado y salta al `catch`; si no, imprime "Error en tiempo de ejecucion" y termina con código 1. Así la rúbrica termina imprimiendo "Error atrapado: indice fuera de rango" igual que el intérprete.

Ejecución en MARS

El `.jar` de MARS 4.5 está en el repositorio (`tools/Mars4_5.jar`), así que las pruebas, el CI y el contenedor usan siempre la misma versión. Lo corremos sin interfaz con opciones para que los códigos de salida distingan un error del programa (1), un fallo de simulación (2) y un error de ensamblado (3). Un detalle que nos tomó tiempo: MARS agrega siempre un salto de línea propio al terminar, y en lugar de ajustar las pruebas lo quitamos en `run_mars`, para que la salida sea exactamente lo que imprimió el programa.

También hicimos que el compilador reconozca cuándo MARS no dejó terminar al programa y lo diga con un mensaje claro, conservando la salida impresa hasta ese momento (Tabla 4). La Figura 8 muestra un `while (true)` que MARS corta a los 10 millones de instrucciones.

Tabla 4. *Problemas que se reportan al ejecutar en MARS.*

Código	Significado
M901	Error del programa sin try (código de salida 1)
M902	Posible ciclo infinito: no terminó en 10 000 000 instrucciones
M903	Posible ciclo infinito: superó el tiempo límite de 10 segundos
M904	Posible recursión infinita: se desbordó la pila
M905	Sin memoria: el heap se agotó
M906	Otro fallo de la simulación

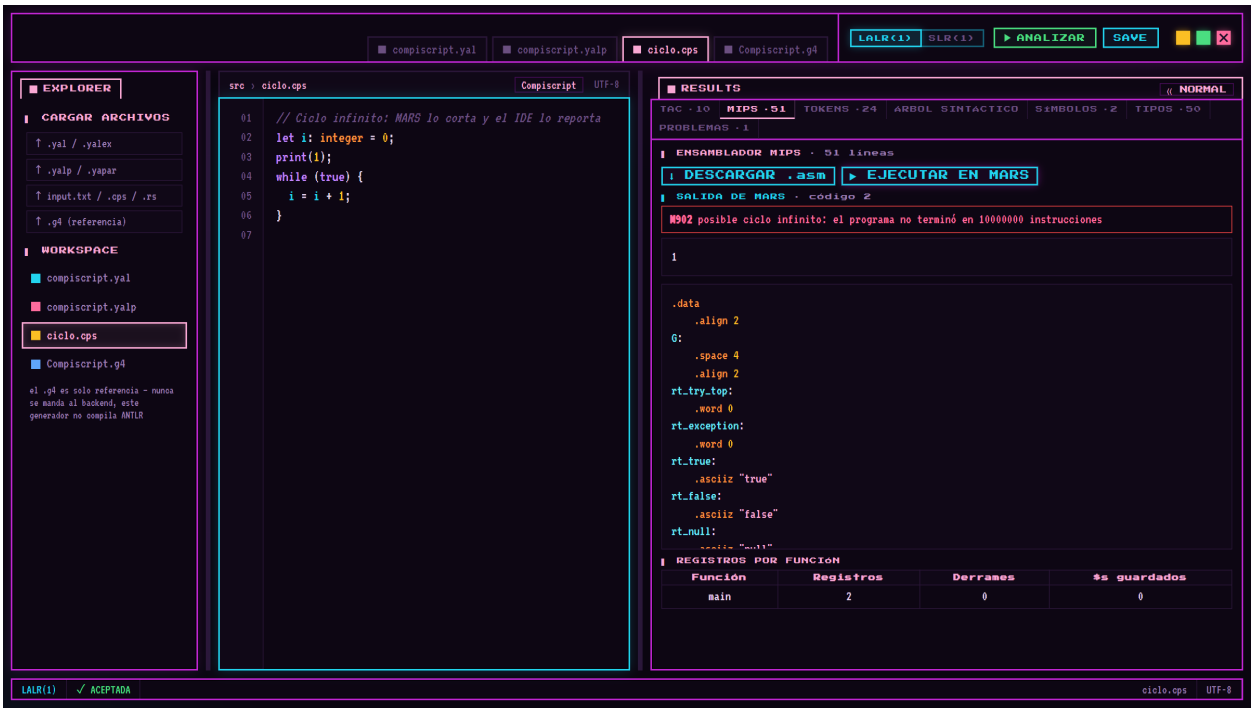


Figura 8. *Un ciclo infinito ejecutado desde el IDE: se conserva la salida impresa (1) y se reporta M902.*

9. Pruebas

Cada fase se probó con su propia batería, y la regla del equipo fue que si dos fases no coincidían en la salida de un programa, mandaba el intérprete de TAC. La batería de MIPS recorre los programas de `workspace/tac/ok` y `workspace/mips/ok`, genera el ensamblador con las dos asignaciones de registros, lo corre en MARS y lo compara con la salida esperada escrita en el propio archivo y con la del intérprete. Para esta última parte escribimos 20 programas del estilo de una entrevista técnica: pirámide de asteriscos, Fibonacci, FizzBuzz, números primos, palíndromos, búsqueda binaria, cinco ordenamientos, torres de Hanoi, lista enlazada, pila y cola, entre otros.

En total la suite tiene **555 pruebas en 44 grupos** y todas pasan. Corre en GitHub Actions en cada push con Java 17 y la variable `MIPS_REQUIRED=1`, que hace que las pruebas de MARS fallen en lugar de saltarse si el simulador no está; cada corrida tarda unos 35 minutos. Además del caso feliz, probamos programas grandes: 8000 líneas compilan en unos 7 segundos, y un anidamiento exagerado da un error P002 en lugar de tumbar el servidor.

10. Discusión

La decisión que más nos ayudó en la última fase fue tener el intérprete de TAC como oráculo. Como ya se comportaba como MIPS, cualquier diferencia con MARS apuntaba directo al traductor, y pudimos comparar cientos de salidas automáticamente sin revisarlas a mano. Lo mismo pasó con la asignación por pila: hacerla primero parecía trabajo extra, pero nos dio una base correcta contra la cual medir `getReg()`.

Pasarle al compilador programas reales, incluso archivos que no eran Compiscript, nos hizo encontrar fallas que las pruebas no cubrían: funciones sin `return` que pasaban, mensajes de sintaxis con listas interminables, comentarios que se cortaban con un acento y un servidor que se quedaba sin memoria con programas largos porque la traza del parser crecía con el cuadrado del tamaño. Todo eso lo corregimos con su prueba, sin ajustar las pruebas existentes para que pasaran.

Quedan limitaciones conocidas. Compiscript no tiene números de punto flotante, así que la fase de MIPS no los soporta (error M001). Una expresión con miles de términos encadenados compila, pero lento, porque los recorridos del árbol son recursivos. Y el CI ya usa unos 35 de sus 45 minutos, así que con más pruebas habrá que subir ese límite o dividir la suite.

11. Conclusiones

- Logramos un compilador completo de Compiscript a MIPS en Rust, con generadores léxico y sintáctico propios: el AFD mínimo tiene 153 estados y la gramática LALR(1) 264 estados sin conflictos.
- Respetar el registro de activación en MIPS, con el valor de retorno en la pila y los enlaces de control y de acceso, hizo que la traducción de llamadas, funciones anidadas y métodos virtuales fuera directa a partir del TAC.
- getReg() redujo los accesos a memoria de la rúbrica de 158 lw y 137 sw a 93 lw y 108 sw, sin cambiar la salida, lo que confirma lo que plantea el libro sobre los descriptores de registros y direcciones.
- Usar el intérprete de TAC como oráculo y ejecutar MARS desde las pruebas, el CI, la consola y el IDE nos dio confianza en que el ensamblador generado, además de ensamblar, da la salida correcta.

- Repartir el trabajo con un plan que fijaba qué recibe y qué entrega cada fase nos permitió avanzar en paralelo; si lo hiciéramos de nuevo, escribiríamos ese plan desde la primera fase.

12. Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.a ed.). Addison-Wesley.

Vollmar, K. y Sanderson, P. (2014). *MARS (MIPS Assembler and Runtime Simulator), versión 4.5*. Missouri State University.

The Rust Project Developers. *The Rust Programming Language*. https://doc.rust-lang.org/book/

Universidad del Valle de Guatemala. Enunciado del proyecto Compiscript y gramática Compiscript.g4, curso Construcción de Compiladores.

Repositorio del proyecto: https://github.com/Vann06/Proyecto_Lexer_Gen